

Explicit Type Conversions

Department of Computer Science
University of Pretoria

15 April 2024

- Type conversions were introduced in Study Unit 3 where the focus was on **implicit type conversions**.
- Remember, implicit type conversions are done by the **compiler**.
- For example, when a value of one type is being assigned to a variable of another type.

- **Explicit type conversions** are when you explicitly instruct the compiler to convert a value from one data type to another.
- This process is also called **type casting** and it is **user-defined**.
- This is useful when there's a need to convert between incompatible types or when precision loss is acceptable.

Possible scenarios where explicit type conversion is needed:

- Consider a return type of float and two int's as input for a division function. If the function divides to the 2 ints, then integer division is applied. Change one of the input parameters to a float to ensure floating point division
- When a return type is not compatible with the type of the variable to which the return value of the function is assigned.

When do we use explicit type conversions?

- Converting Between Primitive Types
- Enforcing Type Conversions
- Avoiding Implicit Conversions
- Casting Pointers (to be covered later)
- Casting Away Constness (to be covered later)

In C++ you can do explicit type conversions in two ways namely:

- Converting by **C-style casting**
- Converting using a **cast operator**

Converting by C-style casting

- This process involves clearly specifying the desired type directly before the expression enclosed in parentheses.
- The syntax follows the pattern:
(type) expression
- Here, 'type' represents the intended data type for the resulting value after conversion.

Consider the following example:

```
#include <iostream>
using namespace std;

int main()
{
    float num = 5.564;

    //C-style casting
    int result = (int)num + 2;

    cout << "The result is = " << result;

    return 0;
}
```

When the above program is run what is the output?

Conversion using Cast Operator

A cast operator is a **unary operator** that compels one data type to be transformed into another data type. C++ supports four types of casting:

- Static Cast
- Dynamic Cast
- Const Cast
- Reinterpret Cast

The dynamic cast, const cast, and reinterpret cast types are used for specific scenarios like casting pointers, dealing with inheritance, etc.

Code Example:

```
#include <iostream>
using namespace std;

int main() {
    // Converting from int to double
    int intValue = 10;
    double doubleValue = static_cast<double>(intValue);
    cout << "Converted value: " << doubleValue << endl;

    // Converting from double to int
    double doubleNumber = 3.14;
    int intNumber = static_cast<int>(doubleNumber);
    cout << "Converted value: " << intNumber << std::endl;

    return 0;
}
```

Documenting functions

When writing functions, and especially more complex functions, it is advisable to **document the functions** for your reference at a later point in time or for someone else who is using your function so that they understand what the function does, the parameter and return types of the function as well as the limitation of the function.

- The documentation of the functions will be placed in the header file where the functions are defined.
- This is typically the file that will be provided to a programmer when they are using a library. In this lecture, we will describe the following:
 - 1 Function definition (Function Descriptions)
 - 2 Restrictions placed on and by the function (Pre- and Post-conditions)

The function descriptions explain what the function does, its purpose, parameters (if any), return type, and any side effects.

Function descriptions will include the following:

- short textual description of the function
- description of the formal parameter list
- description of the return type

To ensure that all functions are documented in the same manner, the following comment should be placed before each function defined in the header file:

```
/**  
A short description of the functionality the function provides  
Parameters: A list of the types of the parameters and what each  
             parameter is used for, if not obvious.  
Return type: The type being returned and a short description of  
             the meaning of the type.  
**/  
}
```

Function Description Example:

```
/**  
Description: Calculates the square of an integer.  
Parameters: A single integer parameter  
Return type: An integer  
**/  
int calculateSquare(int num) {  
    return num * num;  
}
```

The function definition (`calculateSquare`) is described, including its purpose (calculates the square of an integer), parameters (`num`), return type (`int`).

Pre- and Post- Conditions

- Preconditions and postconditions are a method of specifying what a function accomplishes.
- The precondition indicates what must be true before the function is called.
- The postcondition indicates what will be true when the function finishes its work.

Pre- and Post- conditions Example:

```
/**  
Description: Calculates the square of an integer.  
Parameters: A single integer parameter  
Return type: An integer  
Pre-condition: The integer parameter must be  $\geq 0$   
Post-condition: The result must be a square of num  
**/  
int calculateSquare(int num) {  
    return num * num;  
}
```

Pre-conditions and post-conditions are included in the function description as shown above.

Points to note:

- Pre-conditions are conditions that **must be true before** the function is executed.
- These conditions define the expectations and requirements for calling the function. They ensure that the function can execute correctly and safely.
- Pre-conditions often include **constraints** on input parameters, such as valid ranges or non-null pointers.
- Post-conditions are conditions that **must be true after** the function has been executed. They define the state or behavior that the function guarantees to achieve upon completion.
- Post-conditions ensure that the function has performed its task correctly and has met its intended purpose. They may include guarantees about the output values, side effects, or changes to the program's state.

Write a C++ program to calculate the area of a circle. The program must input the radius from the user, then call a function called, "calculateCircleArea", which must then take in the radius value and calculate the area of the circle, and return the result back to the main program. The program must then print out the result of the calculation.

Ensure that your program has a function definition, as well as parameters, pre- and post- conditions explicitly described.

You can either define your function in the Main Function or in header files.

Exercise Solution:

```
#include <iostream>
using namespace std;
/** Function Description: Calculates the area of a circle given
    its radius.
    Parameters: A single double parameter
    Return type: A single double parameter
    Pre-condition: The double parameter must be  $\geq 0$ 
    Post-condition: The result must be the area of a circle */
//define constant
const double pi = 3.14159;
//function of type double
double calculateCircleArea(double radius) {
    return pi * radius * radius;
}
int main() {
    double radius;
    cout << "Enter the radius of the circle: ";
    cin >> radius;
    double area = calculateCircleArea(radius);
    cout << "The area of the circle is: " << area << endl;
    return 0;
}
```